AI-Generated Text Detection with DAIGT V2 and HC3

A Literature Survey, Research-Gap Analysis, Dataset Critique, and Assessment of an Existing Project

Prepared for: Group 02, Section B — Department of Computer Science and Engineering, American International University-Bangladesh (AIUB)

Datasets in scope: DAIGT V2 Train Dataset (Kaggle) and HC3 — Human ChatGPT Comparison Corpus

Date: 21 August 2026

How to read this document

This report answers four questions:

1. **What has already been done** with the DAIGT V2 and HC3 datasets? (Section 2 — 25 papers, 8+ per dataset)
2. **What gap is left** for you to work on? (Section 3)
3. **What is wrong with these two datasets** that you must acknowledge in a paper? (Section 4)
4. **Can your existing NLP course project be turned into a paper?** (Section 5)

Every paper listed has been checked to actually exist and to actually use the dataset it is filed under. Where a source is *not* peer-reviewed (a Kaggle write-up, a GitHub project, a course report, a thesis), it is labelled clearly so you do not accidentally cite it as if it were a journal paper.

Section 6 is a summary table of all papers. Section 7 is the reference list with links.

---

1. Executive summary

Here is the short version of everything below.

On the literature. Both of your datasets are well studied. Across all 25 works surveyed, detection *in-distribution* (train and test on the same dataset) is essentially a solved problem: reported accuracy and F1 sit between 97% and 100%. The HC3 authors themselves reported 99.82 F1 in the original paper back in 2023. Winning Kaggle DAIGT solutions sit around 0.96–0.99 AUC. What this means for you is blunt: **another 99% in-distribution number is not publishable.** Reviewers have seen it thirty times.

On the gap. Every paper that tests *outside* its training distribution shows the same thing — performance collapses. Change the domain, change the generator model, or add simple noise, and detectors that scored 99% drop to 30–80%. That collapse is the open problem, and it is the one you should write about.

On the datasets. Both have documented, citable flaws. DAIGT V2 is essay-only, class-imbalanced (roughly 61% human / 39% AI), assembled from old generator models (the GPT-3.5 / Llama-2 / Falcon / Claude-v1 era), and leaks its labels through surface details like typos and whitespace. HC3 contains output from exactly one generator — the early-2023 ChatGPT (GPT-3.5-Turbo-0301) — is restricted to question-answer pairs, and has a notorious artifact where human answers contain a space before punctuation and ChatGPT answers do not. A detector can score 99% on HC3 by learning that one space.

On your project. Your NLP final project is technically strong — genuinely stronger engineering than several of the published papers below. But as it stands it is **not a paper**, because its research question ("do transformers beat classical baselines?") was answered in 2023 and its results are in-distribution only. The good news is that roughly 70% of the work you would need for a real paper is already built: the splits, the caching harness, the seed-robustness protocol, and the contamination audit are all reusable. Section 5 explains exactly what to keep, what to drop, and what to add.

Our single recommendation. Write a paper about **cross-dataset generalization, artifact leakage, and adversarial robustness**, using your existing infrastructure. Concretely: train on HC3, test on DAIGT and vice versa; then clean the surface artifacts and re-measure; then attack the inputs with typos, homoglyphs, and paraphrasing and re-measure again. Report how much of that famous 99% survives. That is novel, honest, achievable on an RTX 4060, and directly builds on what you already have.

---

2. Papers that used these datasets

2.1 A note on the word "DAIGT"

Be careful here, because it will bite you when you write your related-work section. **"DAIGT" refers to two different things** in the literature:

1. **Your dataset** — the Kaggle competition *LLM: Detect AI Generated Text* (2023–24) and the community-built `DAIGT V2 Train Dataset` by the Kaggle user "thedrcat". This is essays.
2. **M-DAIGT** — a newer shared task at RANLP 2025 on *Multi-Domain Detection of AI-Generated Text*, built from news articles and arXiv abstracts. This is a completely separate dataset.

They share a name and nothing else. Papers A7–A9 below use M-DAIGT, not your data. Cite them only as "related work in the broader DAIGT family" — never claim they used your dataset.

2.2 Group A — Papers using the DAIGT / Kaggle competition data

---

A1. Kaggle "LLM — Detect AI Generated Text", 1st Place Solution *Ranjan Biswas et al., 2024. Kaggle competition write-up (NOT peer-reviewed).* Code: https://github.com/rbiswasfc/llm-detect-ai Competition: https://www.kaggle.com/competitions/llm-detect-ai-generated-text

Dataset: DAIGT V2 plus the PERSUADE corpus, plus adversarial essays they generated themselves.

What they did. Three things stacked together. First, they fine-tuned DeBERTa-v3-large using a *ranking loss* rather than plain classification — instead of asking "is this AI?", the model learns to order documents by how AI-like they are. Second, they trained a *contrastive embedding model*: a model that maps essays into a vector space where similar-origin essays sit close together, then used nearest-neighbour retrieval on that space. Third — and this is the clever part — they fine-tuned several open LLMs on the PERSUADE corpus so those LLMs would write like actual school students, then used those "student-like" AI essays as hard training examples. Finally they blended all of it with classical TF-IDF models.

Results. Top of the leaderboard, roughly 0.99 private-leaderboard AUC.

Limitations. Enormous compute — the repository states training used **4× NVIDIA A100 40GB** (or 4× A6000 48GB). You cannot reproduce this on a 4060, and you should not try. It is also heavily tuned to the competition's hidden test set, which used only seven specific essay prompts (the `RDizzl3_seven` flag in the dataset). That makes it prompt-specific rather than general.

Future work. Generalize beyond the essay domain and beyond the specific generators used.

Why it matters to you. Cite it as the state of the art on this dataset and as evidence of how much compute the top results required. Then position your work as the opposite: what can be learned with modest compute and honest evaluation.

A2. Exploiting Machine Learning Model Ensemble for AI-Generated Texts Detection *C. Zhou, 2024. Transactions on Computer Science and Intelligent Systems Research, vol. 5 (AIDML 2024).* https://wepub.org/index.php/TCSISR/article/view/2382

Dataset: DAIGT V2 Train Dataset — specifically 25,996 human essays from the PERSUADE corpus and 19,509 LLM-generated essays.

What they did. No transformers at all. They used a Byte-Pair Encoding tokenizer feeding TF-IDF character n-grams of length 3 to 5, then trained four classical models — Multinomial Naive Bayes, an SGD classifier, LightGBM, and CatBoost — and combined them with a weighted voting ensemble.

Results. The ensemble beat every individual model. CatBoost was the single best at resisting overfitting.

Limitations. Purely lexical — the model only ever sees word and character frequencies, never meaning or structure. Evaluated on one dataset only, with no cross-domain or adversarial testing at all.

Future work. Use larger and more diverse training data; weight CatBoost more heavily in the ensemble.

Why it matters to you. This is your **classical baseline reference**. It proves that TF-IDF plus tree ensembles is competitive on DAIGT, which is exactly the comparison your project already makes.

A3. Adaptive Ensembles of Fine-Tuned Transformers for LLM-Generated Text Detection *2024.*

arXiv:2403.13335. https://arxiv.org/abs/2403.13335

Dataset: DAIGT, at roughly a 2:1 human-to-AI ratio.

What they did. Built ensembles of several fine-tuned transformer detectors, where the ensemble weights *adapt* rather than being fixed in advance. Tested both in-distribution and cross-domain.

Results. Strong in-distribution. Noticeable degradation out-of-distribution.

Limitations / future work. The adaptive weighting itself becomes unreliable when the test distribution shifts. Improving weight robustness under shift is the stated open problem.

Why it matters to you. Direct evidence, on your dataset, that ensembling does not solve generalization. This also supports the negative ensemble result your own project already found.

A4. AI Generated Text Detection ⭑ MOST RELEVANT PAPER IN THIS REPORT *Alikhanov, Amangeldi, Demeubay, Akhmetzhan, Polat, Zharas, Moldakhmetov (Nazarbayev University), 2026. arXiv:2601.03812*. https://arxiv.org/abs/2601.03812

Dataset: HC3 (~74k samples) merged with DAIGT V2 (44.8k) — 124,195 samples across 20 topics. Exactly your two datasets.

What they did. This is the important design choice: instead of splitting randomly, they used a **topic-based split**. Entire topics were assigned wholly to train, validation, or test. This prevents the model from memorizing topic vocabulary and then "recognizing" the same topic at test time. They compared TF-IDF + Logistic Regression, a BiLSTM, and DistilBERT. Hardware was a single RTX 5080 (16 GB).

Results. Logistic Regression 82.87% accuracy; BiLSTM 88.86% (ROC-AUC 0.94); DistilBERT 88.11% (ROC-AUC 0.96).

Read those numbers again. Everyone else reports 99% on these datasets. These authors report 82–89%. The difference is not that their models are worse — it is that their **evaluation is harder and more honest**. The moment you stop letting the model see the same topics in training and testing, roughly ten points of accuracy evaporate.

Limitations (stated by the authors). Limited dataset diversity and limited compute. They explicitly note that HC3 contains only one AI model.

Future work (stated). Expand dataset diversity; use parameter-efficient fine-tuning such as LoRA; explore smaller distilled models and hardware-aware optimization.

Why it matters to you. This paper is the closest existing work to what you are planning, and you **must** cite it and differentiate from it. Read it first. The good news: they did *not* do strict one-way cross-dataset transfer (train HC3 → test DAIGT), they did *not* audit surface artifacts, and they did *not* run adversarial attacks. Those three things are still open, and they are exactly what Section 3 recommends you do.

A5. A Multirepresentation Stacked Ensemble for AI-Generated Text Detection *Md. Siam Ansary, 2026. International Journal of Intelligent Systems (Wiley)*. https://onlinelibrary.wiley.com/doi/10.1155/int/3992539

Dataset: Two public benchmarks from the DAIGT family.

What they did. A "stacked" ensemble — meaning a second-level model learns how to combine the first-level models, rather than simple averaging. The inputs to the stack are four different views of each document: transformer embeddings, contrastive semantic representations, handcrafted linguistic features, and meta-features derived from an LLM. Mutual-information feature selection trims the feature set, and logistic regression acts as the meta-classifier.

Results. 98.74% accuracy / 0.997 AUC on the first benchmark; 97.92% / 0.994 AUC on the second.

Limitations / future work. A heavy, complicated pipeline. The authors acknowledge robustness to paraphrasing as an unsolved challenge.

Why it matters to you. Directly relevant to your ensemble work. Note the contrast with your own finding: a *learned* meta-classifier (stacking) works, while *fixed-weight* soft voting did not help you. That is a real and reportable observation.

A6. Fast, Interpretable AI-Generated Text Detection Using Style Embeddings *Socolof & Kacholia, Stanford CS224N final project (NOT peer-reviewed)*. https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/final-projects/GiuliaZoeSocolofRitikaKacholia.pdf

Dataset: DAIGT and HC3, plus WELFake and MixSet.

What they did. Rather than fine-tuning a transformer end-to-end, they pre-computed *style embeddings* (vectors trained to capture writing style independent of topic) and then trained tiny classification heads on top. The point is speed and interpretability.

Results. Extremely fast — over 500,000 texts per second once embeddings are cached — with competitive AUC on both DAIGT and HC3.

Limitations. Style embeddings work poorly on mixed human-AI text.

Why it matters to you. A cheap, 4060-friendly architecture that uses both your datasets. Use it as an engineering reference, not a citation of record.

A7. M-DAIGT: A Shared Task on Multi-Domain Detection of AI-Generated Text *Lamsiyah et al., 2025. RANLP 2025. arXiv:2511.11340.* https://arxiv.org/abs/2511.11340 · Proceedings: https://aclanthology.org/2025.ranlp-mdaigt.pdf

Dataset: ⚠ M-DAIGT — a different dataset from yours. 7,000 CNN news articles and 7,000 arXiv abstracts, with AI text from GPT-4o, GPT-3.5, LLaMA-3.2, Qwen2.5, and Mistral.

What they did / results. Ran a shared task. Participating systems reached near-perfect or literally perfect F1. Fine-tuned RoBERTa, ELECTRA, and DeBERTa dominated; adding stylometric features helped.

Limitations. The task is essentially saturated in-domain.

Why it matters to you. Two reasons. First, it shows what a *modern* generator dataset looks like — useful if you want to test whether a 2023-trained detector catches 2025-era models. Second, its saturation is more evidence that in-domain detection is no longer an interesting research question.

A8. A Multi-Strategy Approach for AI-Generated Text Detection *Zain, Farooqui, Rafi, 2025. RANLP 2025 (M-DAIGT). arXiv:2509.00623.* https://arxiv.org/abs/2509.00623

Dataset: M-DAIGT (again, not your dataset).

What they did. Three parallel strategies: (1) fine-tuned RoBERTa-base; (2) TF-IDF plus SVM; (3) a system called "Candace" that extracts probabilistic features from several Llama-3.2 models and feeds them to a custom Transformer encoder.

Results. RoBERTa reached 99.99% accuracy and F1 on Subtask 1 and 100% on Subtask 2. TF-IDF + SVM reached 97.9%.

Why it matters to you. Look at the gap between the fancy method and the simple one: 99.99% versus 97.9%. Two percentage points. On a saturated benchmark, architecture barely matters. Another argument for changing the question rather than the model.

A9. AI-Generated Text Detection Using DeBERTa with Auxiliary Stylometric Features *Yadagiri, Sai Teja, Pakray, Chunka, 2025. RANLP 2025 (M-DAIGT).* https://aclanthology.org/2025.ranlp-mdaigt.2/

Dataset: M-DAIGT.

What they did. DeBERTa combined with auxiliary stylometric features (sentence length, punctuation patterns, function-word ratios, and so on) for binary classification across news and academic domains.

Results. Near-perfect in-domain.

Future work. Cross-domain robustness.

A10. Analyzing AI-Generated Text Using Machine Learning, Deep Learning and Large Language Models *S. Arun, S. Purohit, J. Pareek, 2026. Springer.* https://link.springer.com/chapter/10.1007/978-3-032-00350-8_13

Dataset: DAIGT V2 and several DAIGT variants (External, Proper, V3, V4, plus Gemini-Pro-generated essays).

What they did. A broad three-way comparison across classical ML, deep learning, and LLM-based detection on DAIGT-family data.

Limitations / future work. Generator coverage and generalization remain open.

Why it matters to you. A peer-reviewed survey-style comparison on your exact dataset. Good for framing your related-work section.

A11. DAIGT — Catch the AI *zeyadusf, graduation project (NOT peer-reviewed).* https://github.com/zeyadusf/DAIGT-Catch-the-AI

Dataset: DAIGT (Kaggle plus HuggingFace merges).

What they did. An ensemble of RoBERTa and DeBERTa joined by a feed-forward ReLU fusion layer.

Why it matters to you. An undergraduate project of comparable scope to yours. Useful as a sanity check on what an undergraduate team can build — and as evidence that this level of work exists in abundance, which is why you need a different research question.

A12. Did AI Write This? Using classical machine learning *Benedict Neo / bitgrit, industry blog post (NOT peer-reviewed).* https://medium.com/bitgrit-data-science-publication/did-ai-write-this-7963a8d7ed2d

Dataset: DAIGT V2, filtered to the `RDizzl3_seven` prompts.

What they did. TF-IDF with Logistic Regression, SGD, and XGBoost, evaluated by AUC.

Why it matters to you. A clean tutorial-level baseline replication. Do not cite it in a paper; it is fine as an implementation reference.

2.3 Group B — Papers using the HC3 dataset

B1. How Close is ChatGPT to Human Experts? Comparison Corpus, Evaluation, and Detection ⭑ THE HC3 PAPER — MANDATORY CITATION *Guo, Zhang, Wang, Jiang, Nie, Ding, Yue, Wu, 2023. arXiv:2301.07597.* https://arxiv.org/abs/2301.07597 · Code: https://github.com/Hello-SimpleAI/chatgpt-comparison-detection

Dataset. This paper *introduced* HC3. English has five splits: `reddit_eli5`, `open_qa` (WikiQA), `wiki_csai`, `medicine`, and `finance`. Chinese has six: baike, law, nlpcc_dbqa, medicine, finance, open_qa. The English side has 24,322 human questions and 58,546 answers, of which 26,903 are ChatGPT answers. (You will see slightly different counts in other papers — some count questions, some count answers, some count paired samples, and some use the filtered version. Always state which version and count *you* used.)

What they did. Three detectors: - **GLTR** — a logistic regression over "GLTR Test-2" features. The idea: run the text through a language model and check, for each token, whether it ranked in that model's top 10 / top 100 / top 1000 predictions. AI text uses high-ranked (predictable) tokens more often than humans do. - **RoBERTa-single** — a standard classifier that sees only the answer. - **RoBERTa-QA** — the same classifier but shown the question and the answer together.

Results (from the paper's own tables). - RoBERTa reaches **99.82 F1** on English full-text — in-distribution. - RoBERTa-QA averages **97.48% F1** on English filtered-full text, beating the answer-only model by 5.63 points. - Best Chinese average: **94.22%**. - RoBERTa is far more robust than GLTR when text is split into single sentences: RoBERTa loses about 1.5–2 points, GLTR loses more than 10. - Full documents are much easier than single sentences. A model trained on sentences generalizes better than one trained only on full text (train-on-full → test-on-sentence gives just 81.89 F1, versus 98.43 when trained on sentences). - Per-source results vary wildly. `reddit_eli5` hits a perfect 100.00 F1. But `open_qa` at the sentence level collapses — the ChatGPT-class F1 falls as low as 26.78. - In a human "Turing test", expert annotators reached 0.90 detection accuracy on English.

Limitations (stated by the authors, Section 7). 1. The amount and range of data is still insufficient and unbalanced across sources. 2. **All ChatGPT answers were generated without special prompts.** So a prompt such as "answer in the style of a Reddit user" or "pretend you are Shakespeare" can bypass detectors trained on this data. 3. ChatGPT is mainly English-trained, so conclusions drawn from HC3-Chinese may be imprecise.

Future work (stated). Collect more balanced, multi-style, multi-source, multi-language data; handle style-disguised generation; apply out-of-distribution and anomaly-detection algorithms — which the authors explicitly leave for future work.

Why it matters to you. You must cite this. Note also that the authors themselves flagged OOD detection as future work in early 2023 — and as of 2026 it is still not solved. That is your opening.

B2. HC3 Plus: A Semantic-Invariant Human ChatGPT Comparison Corpus *Su et al., 2023. arXiv:2309.02731.* https://arxiv.org/abs/2309.02731

Dataset: HC3 plus new semantic-invariant tasks — summarization, translation, and paraphrasing. This paper is also where the generator is confirmed in writing: the dataset considers only "the current version of ChatGPT, i.e. GPT-3.5-Turbo-0301."

What they did. Their key insight is that HC3 is question-answering, where the AI generates content freely. But many real-world uses are *semantic-invariant*: the AI is given text and must preserve its meaning while changing the words. They built such tasks and showed detection is much harder there. They then built a detector on Tk-Instruct called InstructDGGC.

Results. InstructDGGC beats the RoBERTa-HC3 baseline, reaching roughly 91.73% overall English accuracy on the harder tasks — compare to 99.82% on plain HC3.

Limitations. Still ChatGPT-only.

Why it matters to you. This is your citation for "HC3's QA format makes the task artificially easy." A drop from 99.8% to 91.7% just from changing the task type is a strong, quotable number.

B3. Towards a Robust Detection of Language Model Generated Text: Is ChatGPT that Easy to Detect? ⭑ BEST TEMPLATE FOR YOUR ROBUSTNESS EXPERIMENTS *Antoun, Mouilleron, Sagot, Seddah (Inria), 2023. TALN/CORIA 2023. arXiv:2306.05871.* https://arxiv.org/abs/2306.05871 · Code: https://gitlab.inria.fr/wantoun/robust-chatgpt-detection

Dataset: HC3 English plus a French translation produced with the Google Cloud Translation API.

What they did. Fine-tuned RoBERTa and ELECTRA for English, CamemBERT and CamemBERTa for French, and XLM-R multilingually. Then they attacked the inputs three ways: - **Misspellings (`+ms`)** — inject realistic typos. - **Homoglyphs (`+hg`)** — swap characters for visually identical ones from other alphabets (a Latin "a" for a Cyrillic "a", for example). A human sees no difference; the tokenizer sees a completely different token. - **A hand-crafted adversarial set** — 61 human-written answers deliberately composed in ChatGPT's didactic, list-heavy explaining style.

Results. This is the paper to quote when you need to show how fragile these detectors are: - In-domain F1 up to **99.88** (English RoBERTa-QA). Textbook-perfect. - Under misspelling attack, ChatGPT-class recall falls to 0.79 (F1 0.88). - Under homoglyph attack, F1 falls to 0.93. - On the hand-written adversarial human set, raw accuracy drops to **33.57%** (CamemBERTa) and 59.12% (XLM-R). A detector scoring 99.88% in-domain does *worse than a coin flip* on humans who write like ChatGPT. - On Bing-generated text with misspellings, accuracy falls to **44.81%** and **28.18%**. - Adversarial training — adding 50% misspelled and 50% homoglyph examples to the training set — substantially restores robustness, bringing BingGPT back to around 91%.

Limitations (stated). Strong results are in-domain only and "do not generalize in out-of-domain scenarios." The detector "relies heavily on the didactic response style of ChatGPT." Robustness was tested only against basic attacks. The French set is machine-translated, so it carries translation artifacts.

Future work (stated). Extend the adversarial dataset; find a general robustness approach instead of an endless "cat-and-mouse game"; translate the datasets to build detectors in other languages.

Why it matters to you. Copy this experimental design. It is cheap — typo and homoglyph injection are twenty-line scripts — and it produces dramatic, publishable numbers.

B4. Multiscale Positive-Unlabeled Detection of AI-Generated Texts ⭑ YOUR CITATION FOR THE HC3 WHITESPACE ARTIFACT *Tian, Chen, Ho et al., 2024. ICLR 2024 Spotlight. arXiv:2305.18149.* https://arxiv.org/pdf/2305.18149 · Code: https://github.com/YuchuanTian/AIGC_text_detector

Dataset: HC3 (including the HC3-English-Sent short-text version) and TweepFake.

What they did. Short texts are genuinely ambiguous — a five-word sentence may be indistinguishable whether a human or a machine wrote it. Forcing a hard AI/human label on such text teaches the model noise. Their solution frames short machine texts as partly "unlabeled" using **Positive-Unlabeled learning**, a technique for training when you have confident positives but uncertain negatives.

Results. RoBERTa-MPU reaches 85.31 F1 on HC3-English-Sent, versus 58.60 for plain RoBERTa — a 27-point gain on short text. Also 91.4% accuracy on TweepFake.

The critical part for you. In building this, the authors discovered and documented that **in HC3-English, human answers contain extra spaces before punctuation and ChatGPT answers do not.** A detector can achieve near-perfect scores by learning to count spaces. They released a cleaning kit to remove this. If you report HC3 results without cleaning this artifact, a reviewer who knows this paper will reject you.

Future work. Better short-text and cross-domain detection.

B5. Are AI-Generated Text Detectors Robust to Adversarial Perturbations? *2024. ACL 2024. arXiv:2406.01179.* https://arxiv.org/pdf/2406.01179

Dataset: HC3 in-domain (26,903 human / 58,546 ChatGPT) plus TruthfulQA cross-domain.

What they did. Systematic character-level perturbations — swapping adjacent characters, dropping characters, simulating keyboard slips, and inserting characters — then proposed a more robust detector.

Results. Standard detectors degrade sharply under perturbation; the proposed method holds up better.

Future work. Broader attack coverage.

Why it matters to you. A peer-reviewed ACL citation for the exact attack family you should run. Pair it with B3.

B6. LLM-Detector: Improving AI-Generated Chinese Text Detection with Open-Source LLM Instruction Tuning *2024. arXiv:2402.01158.* https://arxiv.org/pdf/2402.01158

Dataset: HC3 (12,853 questions), regenerated with nine different LLMs including GPT-4, plus the M4 dataset.

What they did. Instead of fine-tuning a classifier head, they instruction-tuned open-source LLMs to *act* as detectors, at both document and sentence level.

Results. Strong Chinese detection, and notably better generalization across generators than models trained only on original HC3.

Limitation. Chinese-focused.

Why it matters to you. This is proof of concept for a key idea: **regenerating HC3 questions with newer models fixes the single-generator problem.** If you want to test modern LLMs, you do not need a new dataset — you can re-answer HC3 questions with GPT-4o or Llama 3 and reuse the human side as-is.

B7. Detecting Machine-Generated Texts by Multi-Population Aware Optimization for Maximum Mean Discrepancy (MMD-MP) *2024. arXiv:2402.16041.* https://arxiv.org/pdf/2402.16041

Dataset: HC3 and XSum, with machine text from ChatGPT, GPT-2, GPT-3, GPT-Neo, and GPT4All-J.

What they did. A different framing entirely. Rather than classifying each document, they run a **two-sample statistical test**: given a *set* of texts, does it come from the "human" population or the "machine" population? Maximum Mean Discrepancy measures the distance between two distributions.

Results. More stable than per-document classifiers when the test data mixes several generators.

Why it matters to you. Worth mentioning in related work as a non-classifier alternative. Probably too involved to implement yourself.

B8. Comparing hand-crafted and deep learning approaches for detecting AI-generated text: performance, generalization, and linguistic insights *R. Ardeshirifar, 2025. AI and Ethics (Springer).* https://link.springer.com/article/10.1007/s43681-025-00699-4

Dataset: HC3 plus the GPT-2 Output Dataset, used as a cross-model test.

What they did. Compared handcrafted linguistic features against deep learning. Importantly, they explicitly standardized punctuation and contractions in order "to eliminate superficial differences between human and AI-generated text" — that is, they deliberately removed the artifacts described in B4.

Results. Deep models win in-domain; handcrafted features are more interpretable; both degrade when tested across generators.

Future work. More generators; robustness.

Why it matters to you. A peer-reviewed precedent for artifact cleaning as a methodological step. Cite it when you justify your own cleaning stage.

B9. Supervised Machine Generated Text Detection Using LLM *Worcester Polytechnic Institute, MS thesis (NOT peer-reviewed).* https://digital.wpi.edu/downloads/hh63t0231

Dataset: HC3, using the question / human-answer / ChatGPT-answer structure.

What they did. Fine-tuned RoBERTa on HC3.

Results. High in-domain accuracy; the thesis discusses vulnerability to paraphrasing attacks.

Why it matters to you. Reference material only. Use it for implementation details, not citation.

B10. Synergizing linguistic features and transformer networks for detecting AI-generated text *2025.*

Knowledge and Information Systems (Springer). https://link.springer.com/article/10.1007/s10115-025-02637-6

Dataset: HC3 English plus M4GT English.

What they did. Combined DistilBERT with explicit linguistic features.

Results. 99.45% accuracy on HC3-English with linguistic features; 96.23% on M4GT.

Why it matters to you. More evidence of HC3 saturation — and note that a *lightweight* DistilBERT gets 99.45%. Your DeBERTa got 99.92%. Neither number distinguishes a good detector from a great one, which is precisely the problem.

B11. Detecting the Machine: A Comprehensive Benchmark of AI-Generated Text Detectors *Baidya et al., 2026. arXiv:2603.17522.* https://arxiv.org/abs/2603.17522

Dataset: HC3 (23,363 pairs across 5 domains, **length-matched**) plus ELI5 with Mistral-7B generations.

What they did. A wide benchmark — classical classifiers, fine-tuned transformers (BERT, RoBERTa, ELECTRA, DistilBERT, DeBERTa-v3), a CNN, an XGBoost stylometric model, perplexity-based detectors, and LLM-as-detector — evaluated across domains and against adversarial “humanization”.

Results. Transformers hit near-perfect scores in-distribution but degrade under domain shift and humanization.

The critical part for you. These authors deliberately **length-matched** HC3 to remove what they call the *length confound*: human answers and ChatGPT answers differ systematically in length, so a detector can score well by essentially measuring word count. If you do not control for length, part of your accuracy is just a ruler.

Future work. Robust cross-domain detection.

B12. Investigating the Influence of Prompt-Specific Shortcuts in AI Generated Text Detection *Kim et al., 2024. arXiv:2406.16275.* https://arxiv.org/pdf/2406.16275

Dataset: HC3, all five splits.

What they did. Showed empirically that detectors trained on HC3 latch onto **prompt-specific shortcuts** — patterns tied to how the data was collected rather than to AI-ness in general. They then designed adversarial instructions (“FAILOpt”) that exploit those shortcuts to fool detectors.

Results. Detectors demonstrably rely on spurious cues.

Why it matters to you. Together with B4 and B11, this is the third independent, citable demonstration that HC3 accuracy is partly fake. Three sources on the same point makes for a very defensible limitations section.

B13. Detecting AI-generated texts using machine learning models *2025. Communications in Statistics: Case Studies, Data Analysis and Applications (Taylor & Francis).* https://www.tandfonline.com/doi/full/10.1080/23737484.2025.2550442

Dataset: HC3 (Guo et al. QA data).

What they did. Bag-of-Words, TF-IDF, and doc2vec vectorization feeding classical ML classifiers.

Results. High in-domain accuracy.

Why it matters to you. A peer-reviewed classical-baseline paper on HC3 — the natural comparison point for your own Naive Bayes / Logistic Regression / SVM numbers.

B14. Large Language Models can be Guided to Evade AI-Generated Text Detection (SICO) *2023. arXiv:2305.10847.* https://arxiv.org/pdf/2305.10847

Dataset: HC3 (the GPT-3.5 detector under attack is trained on it), plus DetectGPT and GPT-2-detector baselines.

What they did. Showed that with the right prompting strategy, ChatGPT can be guided to produce text that slips past HC3-trained detectors.

Results. HC3-trained detectors are readily evaded.

Why it matters to you. Connects back to Guo et al.’s own stated limitation — that all HC3 answers were generated without special prompts. This paper is the demonstration of why that matters.

2.4 Group C — Papers using both datasets

Four of the works above use DAIGT and HC3 together. Grouped here for convenience because these are your most direct comparison points.

Paper	Where	Why it matters
A4 — Alikhanov et al. (arXiv:2601.03812)	Section 2.2	Closest existing work. HC3 + DAIGT V2, topic-based split, 82–89% accuracy.
A6 — Socolof & Kacholia (Stanford CS224N)	Section 2.2	Style embeddings on DAIGT + HC3. Fast and cheap. Not peer-reviewed.
A5 — Ansary (Wiley IJIS 2026)	Section 2.2	Two-benchmark stacked ensemble.
C4 — see below	Here	Neuron-level explanation of why detectors fail OOD.

C4. How to Generalize the Detection of AI-Generated Text: Confounding Neurons *2025. Findings of EMNLP 2025.* https://aclanthology.org/2025.findings-emnlp.1388.pdf

Dataset: DAIGT (out-of-sample) plus **HC3**, XSum, M4, and Ghostbuster (out-of-distribution).

What they did. Looked inside a BERT-based detector to find individual neurons in the feed-forward layers responsible for spurious, dataset-specific features — they call these *confounding neurons*. Then they simply switched those neurons off.

Results. Removing roughly 20 neurons — about **0.05% of the network** — improved out-of-distribution accuracy by up to **6.9%**.

Why it matters to you. This is the mechanistic proof of everything else in this report. Detectors memorize spurious dataset-specific features, and those features are localized enough that deleting a handful of neurons makes the model generalize better. Cite this as the strongest evidence that in-distribution scores on DAIGT and HC3 measure memorization as much as detection.

3. Research gap analysis

3.1 The pattern across all 25 papers

Read the survey again and one shape emerges. Every single paper reports one of two things:

- **In-distribution numbers between 97% and 100%.** (A1, A2, A5, A7, A8, A9, B1, B2, B3, B5, B10, B11, B13)
- **A collapse the moment anything changes.** (A3, A4, B2, B3, B5, B11, B12, B14, C4)

Nobody has published a strong, general detector. What has been published, over and over, is a strong *dataset-specific* detector — and three independent teams (B4 whitespace, B11 length, B12 prompt shortcuts) plus a mechanistic study (C4 confounding neurons) have now shown *why*: the models are learning collection artifacts, not AI-ness.

This is your gap. Not "build a better detector" — that road is crowded and the compute requirements are absurd (see A1's four A100s). Your gap is **measuring honestly what is actually being learned, and how little of it survives contact with reality.**

3.2 Candidate gaps, ranked by feasibility for your team

You have an RTX 4060 and a small team. Here are the realistic options, best first.

Gap 1 — Cross-dataset generalization. ⭑ RECOMMENDED

Train on HC3, test on DAIGT. Train on DAIGT, test on HC3. Report both directions alongside the in-distribution numbers.

Is it novel? Yes, with a caveat. Alikhanov et al. (A4) **merged** the two datasets and used a topic split. Nobody has published a clean, strict one-way transfer matrix between exactly these two datasets. Merging and transferring are different experiments: merging asks "can a model handle both?", transferring asks "does knowledge of one carry to the other?" The second is the harder and more interesting question.

Feasibility: **Very high.** No new data collection. TF-IDF plus Logistic Regression trains in minutes on CPU. DistilBERT and RoBERTa-base fine-tune comfortably in 8 GB at batch size 8 and sequence length 256.

Expected result: A large drop. Probably to somewhere between 50% and 75%. The two datasets differ in domain (essays vs. QA), length (2,216 characters vs. short answers), and generator mix — three simultaneous distribution shifts.

Gap 2 — Adversarial and paraphrase robustness. ⋆ RECOMMENDED AS A COMPANION

Apply typo injection, homoglyph substitution, and paraphrasing to the test sets, and measure the drop.

Is it novel? Partially. B3 did this on HC3; B5 did character perturbations. Nobody has done it on DAIGT V2, and nobody has done it while simultaneously reporting cross-dataset transfer. The combination is new.

Feasibility: **High.** Typo and homoglyph injection are short scripts. For paraphrasing, back-translation through a small MT model (English → German → English) is the cheap option. The DIPPER paraphraser is stronger but heavier — treat it as optional.

Expected result: Following B3's precedent, expect drops of 10–40 points, possibly worse.

Gap 3 — Detecting newer LLMs than the datasets contain.

Neither dataset contains GPT-4o, Claude 3.5 or 4, Llama 3, Gemini, or current Mistral. Generate a few hundred fresh samples with a modern model and see whether your 2023-trained detectors catch them.

Is it novel? Yes for these two datasets specifically. B6 did something similar for Chinese by regenerating HC3 questions with nine LLMs — copy that method.

Feasibility: **High**, with one dependency: you need some API access or a free tier. You only need a few hundred test samples, not training data, so cost is low. Practical approach: take HC3 questions, re-answer them with a modern model, keep the original human answers as the human class.

Expected result: A meaningful drop. Modern models write less formulaically than 2023 ChatGPT, and the stock phrases ("As an AI language model…", "It is important to note…") that detectors rely on have largely disappeared.

Gap 4 — Artifact and shortcut auditing. ⋆ RECOMMENDED AS THE FRAMING

Quantify how much of the 99% comes from whitespace, punctuation, length, and typos rather than from anything meaningful. Clean each artifact, re-measure, and report the loss.

Is it novel? The individual artifacts are documented (B4 whitespace, B11 length, B12 shortcuts). What has *not* been done is a systematic ablation: remove artifact A, measure; remove B, measure; remove both, measure. Nobody has published "here is how much of the reported accuracy on DAIGT and HC3 is real."

Feasibility: **Very high.** Pure preprocessing. Zero extra GPU cost. This is the highest ratio of contribution to effort in the entire report.

Expected result: On HC3 especially, a substantial drop. The whitespace cue alone may account for several points.

Gap 5 — Explainability, calibration, and false-positive analysis.

Which tokens drive the decision? Are the confidence scores calibrated (does a 90% confidence mean right 90% of the time)? What is the false-positive rate on non-native English writing?

Feasibility: **Medium.** Attribution methods add complexity, and the non-native-speaker question needs a corpus you do not have.

Why it still matters: The false-positive angle is the ethically important one — real students get accused of cheating by these systems. If you want an ethics angle for your discussion section, this is it.

Gaps we recommend you avoid. Watermarking (requires control over the generating model), large-scale multilingual bias (needs corpora you do not have), and human-AI hybrid text detection (needs a hybrid corpus; MixSet exists but adds a third dataset and a lot of scope).

3.3 Our recommendation

Combine Gaps 1, 2, and 4 into a single paper.

Proposed title direction: *"How much of AI-text detection accuracy is real? Cross-dataset transfer, artifact leakage, and adversarial robustness on DAIGT V2 and HC3."*

The narrative writes itself:

1. We reproduce the standard 99% result on both datasets. (Confirms our pipeline is correct — and you already have this.)
2. We show that most of it does not transfer between datasets.
3. We show that a measurable share of it comes from surface artifacts, not language.
4. We show that trivial attacks destroy what remains.
5. We conclude that current benchmark numbers substantially overstate real-world detection ability.

That is an honest, useful, publishable contribution that requires no A100s.

3.4 Concrete study design

Models. Four, spanning the cheap-to-expensive range: - TF-IDF + Logistic Regression (classical, CPU, seconds) - TF-IDF + LightGBM (classical, CPU, minutes) - DistilBERT-base (transformer, fits easily in 8 GB) - DeBERTa-v3-base (transformer, your project shows it peaks around 6.18 GiB at sequence length 128 — at 256 you may need batch size 8 with gradient accumulation)

Experimental matrix.

Regime	Train on	Test on	What it tells you
ID-1	HC3	HC3	Baseline; reproduces the literature
ID-2	DAIGT	DAIGT	Baseline; reproduces the literature
X-1	**HC3**	**DAIGT**	**Does QA knowledge transfer to essays?**
X-2	**DAIGT**	**HC3**	**Does essay knowledge transfer to QA?**

Run every cell twice: once on raw text, once on artifact-cleaned and length-matched text. That gives eight numbers per model, thirty-two in total. That is a full results table.

Artifact cleaning protocol. Normalize whitespace before punctuation (cite B4 and use their kit); standardize punctuation and contractions (cite B8); length-match the human and AI classes by sampling or truncation (cite B11); optionally inject typos into the AI class to neutralize the typo cue documented in the Kaggle discussions.

Attack protocol. Apply to test sets only, never to training: - Typo injection at 1%, 5%, and 10% of characters - Homoglyph substitution at 1%, 5%, and 10% - Back-translation paraphrase (English → German → English)

Report accuracy at each attack strength — the resulting degradation curve makes an excellent figure.

Metrics. Always report **accuracy, macro-F1, per-class F1, AUROC, and false-positive rate.** Not weighted F1 alone. DAIGT's 61/39 imbalance means weighted F1 flatters you, and the false-positive rate is what matters ethically — it is the rate at which honest students get flagged.

Sequence length. Use 256 tokens minimum, ideally 512 for DAIGT. Your current 128 truncates 99.6% of DAIGT essays, keeping only 31.3% of the median essay. That was fine as a course constraint; in a paper it is an open invitation for a reviewer to reject you. See Section 5.

---

4. Dataset limitations

This section gives you the material for the limitations section of your paper, with a citation attached to each claim.

4.1 DAIGT V2 Train Dataset

It is a merge, not a curated corpus. `train_v2_drcat_02.csv`, built by the Kaggle user "thedrcat", combines many separate community datasets. The human side is the PERSUADE corpus (25,996 essays). The AI side aggregates: chat_gpt_moth (2,421), llama2_chat (2,421), mistral7binstruct_v2 (2,421), mistral7binstruct_v1 (2,421), original_moth (2,421), train_essays (1,378), llama_70b_v1 (1,172), falcon_180b_v1 (1,055), darragh_claude_v7 (1,000), darragh_claude_v6 (1,000), radek_500 (500), plus Llama-2-7b, Mistral-7B-Instruct, cohere-command (350), palm-text-bison1 (349), and radekgpt4 (200). Nobody applied a single consistent generation protocol across these.

Class imbalance. Roughly 44,868 essays total: about **27,371 human and 17,497 AI**, or 61% / 39%. Alikhanov et al. (A4) note the same skew. Consequence: report per-class F1, not just accuracy or weighted F1.

Essay-only domain. All human text is argumentative essays by US students in grades 6–12, drawn from PERSUADE 2.0, spanning only 15 prompts across 2 writing tasks. A model trained here learns "student persuasive essay versus LLM essay." It has no reason to work on news, code, reviews, chat, or academic writing.

Narrow prompt coverage. The competition's hidden test set used only seven prompts, flagged by the `RDizzl3_seven` boolean. Filtering to those seven boosts leaderboard scores while making the model prompt-specific. This is a documented overfitting trap and is why A1's winning solution does not straightforwardly generalize.

Label noise. Labels come from whichever source dataset a row was taken from, not from manual annotation. Any mislabeled or mixed sample propagates silently into your training set.

Surface artifacts leak the label. The best-known example from the competition: **the hidden test essays contained typos while the training essays did not.** A detector could score well by learning "clean text equals AI." Competitors responded either by spell-correcting the test set or by injecting typos into training — there is a public "AI-Essay-Detection-Daigt-V2-Dataset-with-typos" variant for exactly this reason. AI essays also carry telltale characters (emojis, unusual Unicode) absent from human ones.

Duplication. Merging many community datasets introduces near-duplicate essays. Deduplicate before splitting or you will leak between train and test. (Encouragingly, your own contamination audit found zero leaked rows in DAIGT — see Section 5.)

Generator age. Everything is from the late-2023 generation: GPT-3.5, Llama-2, Mistral-7B, Falcon-180B, Claude v1-era, PaLM, Cohere. There is **no GPT-4o, Claude 3.5 or 4, Llama 3, Gemini, or current Mistral.** As of 2026 this dataset represents a generation of models that is essentially obsolete.

Population bias. PERSUADE writers are school students. The "human" class therefore skews young and non-expert. A detector trained on it may systematically misjudge professional, academic, or non-native-speaker writing — a real false-positive risk with real consequences.

4.2 HC3 (Human ChatGPT Comparison Corpus)

One generator, one moment in time. Every AI answer comes from **GPT-3.5-Turbo-0301** — the early-2023 ChatGPT. The project began roughly ten days after ChatGPT's launch. Su et al. (B2) state this explicitly. Guo et al. (B1) further note that all answers were generated **without special prompts**, so any styled or prompt-engineered generation escapes detectors trained on this data — a weakness B14 then demonstrated in practice.

QA-only format. HC3 is entirely question-answer pairs. Su et al. (B2) showed that detection on semantic-invariant tasks — summarization, translation, paraphrasing — is substantially harder, with accuracy falling from about 99.8% to about 91.7%. HC3's format therefore overstates real-world detectability.

The whitespace artifact. ⚠ The single most important thing to know about this dataset. Tian et al. (B4, ICLR 2024) found that **human answers in HC3-English contain extra spaces before punctuation, while ChatGPT answers do not.** A detector can approach ceiling performance by counting spaces. They released a cleaning kit. Ardeshirifar (B8) independently standardizes punctuation and contractions for the same reason. **If you publish HC3 numbers without addressing this, you will be rejected.**

The length confound. AI and human answers differ systematically in length. Baidya et al. (B11) length-match HC3 specifically to prevent detectors from exploiting this. Without matching, part of your accuracy is a word counter.

Prompt-specific shortcuts. Kim et al. (B12) demonstrate that detectors latch onto cues tied to how the data was collected, and design adversarial instructions that exploit those cues to fool detectors.

Stock phrases and formatting. HC3 ships "filtered" versions with indicating words removed, precisely because ChatGPT's stock phrases ("As an AI language model…", "It is important to note…") and heavy list formatting are dead giveaways. Unfiltered HC3 lets detectors cheat on these.

Saturation. Detectors routinely reach around 99% F1: Guo's own RoBERTa at 99.82, DistilBERT with linguistic features at 99.45 (B10), your DeBERTa at 99.92. At this ceiling the benchmark can no longer distinguish a good detector from a great one.

Poor out-of-distribution transfer. Multiple works (B3; C4) use HC3 as an OOD target and show sharp drops when models trained elsewhere are tested on it, or vice versa.

Collection artifacts in the data itself. Some rows stored as "ChatGPT answers" are actually API failure messages such as "Too many requests in 1 hour." These are trivially separable and artificially inflate scores. **Your own project independently discovered this** — see Section 5.

Chinese-side caveat. Guo et al. caution that ChatGPT is mainly English-trained, so conclusions from HC3-Chinese may be imprecise.

4.3 Combined effect

Put these together and a picture emerges. When a paper reports 99% on DAIGT or HC3, that number is some mixture of:

- genuine detection of statistical differences between human and machine language,
- memorization of the specific generators used in 2023,
- exploitation of whitespace, punctuation, length, typos, and stock phrases,
- topic and prompt memorization.

Nobody has published a clean decomposition of that mixture. **Producing one is a legitimate contribution, and it is exactly what Gap 4 proposes.**

---

5. Assessment of your NLP final project

This section covers Task 4 — whether `NLP_FINAL_PROJECT_GROUP2_MERGED.md` can be used in a paper.

5.1 What the project actually is

A binary human-versus-machine text detection study. BERT (`bert-base-uncased`) and DeBERTa (`microsoft/deberta-v3-base`) fine-tuned on DAIGT V2 and HC3, across a 32-run hyperparameter sweep (2 datasets × 2 models × 8 configs), plus 8 seed-robustness runs — 40 cached runs total. A validation-weighted soft-vote ensemble sits on top, and everything is compared against midterm classical baselines (Naive Bayes, Logistic Regression, SVM over BoW and TF-IDF). Both datasets were balanced to 3,000 documents per class. Total training time: 1.91 hours on an RTX 3060 Ti.

Headline test-set weighted F1:

Model	DAIGT V2	HC3
Naive Bayes (BoW)	0.9566	0.8583
Logistic Regression (BoW)	0.9825	0.9333
SVM (TF-IDF)	0.9875	0.9083
BERT (best config)	0.9908	0.9875
DeBERTa (best config)	**0.9933**	**0.9992**
Soft-vote ensemble	0.9925	0.9992

5.2 The honest verdict

As it stands, this is not a paper. But it contains roughly 70% of the infrastructure for one, and — this matters — it contains three findings that most published papers in this space do not have.

Let me be direct about both halves.

Why it is not a paper yet.

The research question was settled in 2023. “Do fine-tuned transformers beat classical baselines at AI-text detection?” has been answered affirmatively by B1, B10, B13, A2, A5, and a dozen others. A reviewer reading your abstract will know the answer before finishing the first sentence. Novelty is the bar you are currently failing — not rigour.

Everything is in-distribution. Every number in the report comes from training and testing on the same dataset with a random split. Section 3 explains why that is now the least interesting experimental setting in this field.

The artifacts are uncontrolled. Your HC3 result of 0.9992 was almost certainly boosted by the whitespace artifact (B4), the length confound (B11), and stock phrases. You did not clean these — nor should you have, for a course project, since the spec did not ask. But a reviewer who knows B4 will ask, and “we did not check” is not an answer.

The 128-token limit is a serious methodological problem. Your own diagnostics are damning and, to your credit, you reported them: **99.6% of DAIGT essays exceed 128 tokens, and the median essay contributes only 31.3% of its tokens.** Your DAIGT results describe classification from the opening third of an essay. That was a spec constraint. In a paper it is a fatal flaw unless fixed.

The sample is small. 6,000 documents per dataset out of 44,868 and 85,449 available. Defensible for matching the midterm protocol; harder to defend in a paper.

Why it is nonetheless strong work.

Your methodology is more rigorous than several papers in Section 2. Specifically:

You verified the split from first principles. Rebuilding the midterm partition by index-splitting, then asserting order-sensitive equality, then refitting all six classical models and reproducing all 24 midterm metrics to four decimal places — that is a level of care most published work skips entirely.

You ran a seed-robustness study. Three seeds on the winning configuration per model per dataset. You found BERT varying by **0.0267 F1** on DAIGT — wider than most of the differences between your eight configurations. And you drew the correct conclusion: differences smaller than the noise band should not be interpreted. Most papers report a single seed and rank configurations that differ by 0.001.

You ran a contamination audit and it found something real. Eight leaked rows in the HC3 test set (0.67%), several of which "are not answers at all but API failure messages such as 'Too many requests in 1 hour' stored as ChatGPT answers." **That is an independent rediscovery of a genuine HC3 defect.** You also handled it correctly — rather than modifying the split (which would have invalidated the classical comparison), you rescored every model on the clean subset and showed the largest F1 change was 0.0010 with no ranking change.

You reported negative results honestly. The ensemble did not beat DeBERTa. You said so, and then explained the mechanism properly: on HC3, BERT made 15 errors and DeBERTa made 1, with **zero overlap** — a perfect combiner would have scored flawlessly. But any mixing weight large enough to let BERT fix DeBERTa's one error also admits BERT's fifteen. You further noted that the reported weight of 0.00 is not a rejection of BERT but an eleven-way tie across weights 0.00 to 0.50 that a 480-document validation set cannot resolve. That analysis is better than most published ensemble discussions.

You caught non-determinism. DeBERTa reproduced its test F1 exactly on retraining; BERT did not, differing by 0.0058 and 0.0167. Your explanation — that GPU training is not bitwise deterministic, and since the best epoch is chosen by validation F1, a tiny numerical difference can select a different epoch and thus a materially different model, amplified when the validation curve is flat — is exactly right.

The engineering is excellent. Atomic writes, epoch-level checkpointing, cache-on-completion, checkpoint deletion ordered after metrics land, tested by an actual power cycle. Bfloat16 chosen because DeBERTa overflows in float16. The pinned Transformers version because v5 removed `warmup_ratio`. This is production-grade discipline.

5.3 What to keep, drop, and add

Keep — this is your foundation:

Asset	Reuse as
Verified-split methodology	Your reproducibility section, extended to cross-dataset splits
40 cached runs (results + probs)	In-distribution baselines — already done, no recompute needed
Seed-robustness protocol (3 seeds)	Apply to every new experiment; the 0.0267 noise band is your significance threshold
Contamination audit	Promote to a *finding*, not an appendix — it corroborates B11 and B12
Fault-tolerant harness	Run the new experiments without babysitting
Classical baselines (NB / LR / SVM)	The cheap end of your model spectrum in the transfer matrix
Ensemble negative result + analysis	A discussion subsection, reframed against A5's stacking success

Drop:

- The framing "transformers beat classical baselines." Not novel. Demote to a one-paragraph sanity check.
- The 128-token limit. Go to 256 minimum, 512 for DAIGT.
- Weighted F1 as the headline metric. Switch to macro-F1 plus per-class F1 plus false-positive rate.
- The soft-vote ensemble as a headline contribution. Keep it as a discussion point.

Add — this is the actual paper:

1. **The cross-dataset transfer matrix.** Train HC3 → test DAIGT, and the reverse. This is the new core result.
2. **The artifact-cleaning ablation.** Every experiment run twice — raw and cleaned. Report the loss. This is your novelty.
3. **The adversarial stress test.** Typos, homoglyphs, back-translation, at three strengths each.
4. **Optional: the modern-LLM probe.** Regenerate a few hundred HC3 answers with a current model (following B6's method) and measure the drop.

5.4 Suggested timeline

Stage	Weeks	What	Success check
1	1–2	Re-run in-distribution baselines at 256 tokens; confirm the ~99% reproduces	AUROC ≥ 0.98 in-distribution
2	3–5	Cross-dataset transfer matrix, raw and cleaned	Transfer accuracy measured in all four cells
3	6–7	Adversarial attacks at three strengths	Degradation curves plotted
4	8	Optional modern-LLM probe	200–500 fresh samples scored
5	9–10	Write-up	—

Decision rule for Stage 2: if cross-dataset accuracy falls below roughly 70% — which we expect — you have your headline: *these benchmarks do not transfer.* If instead it stays above 90% even after cleaning, that would be a genuinely surprising positive result; in that case pivot the headline to Stage 4, where a drop is near-certain.

If compute gets tight: drop DeBERTa-v3-base and keep DistilBERT plus the classical models. The classical-versus-transformer contrast under distribution shift is itself a finding, and it costs almost nothing to produce.

5.5 One small correction

The report states experiments ran on an **RTX 3060 Ti**. Make sure the hardware statement in any paper matches whatever machine you actually use — this is the kind of detail reviewers check against reported runtimes and memory peaks.

---

6. Summary comparison table

#	Paper	Year / Venue	Dataset(s)	Method	Headline result	Key limitation
A1	Biswas — Kaggle 1st place	2024 / Kaggle	DAIGT V2 + PERSUADE	DeBERTa-v3-large, ranking loss, contrastive, TF-IDF	~0.99 AUC	4×A100; prompt-specific; not peer-reviewed
A2	Zhou — ML ensemble	2024 / TCSISR	DAIGT V2	TF-IDF + NB/SGD/LGBM/CatBoost vote	Ensemble > singles	Lexical only; no OOD test
A3	Adaptive ensembles	2024 / arXiv 2403.13335	DAIGT	Weighted transformer ensemble	High in-dist.	Degrades OOD
A4	**Alikhanov et al.**	**2026 / arXiv 2601.03812**	**HC3 + DAIGT V2**	**LR / BiLSTM / DistilBERT, topic split**	**82.9 / 88.9 / 88.1%**	**Diversity + compute; LoRA future**
A5	Ansary — stacked ensemble	2026 / IJIS (Wiley)	Two DAIGT benchmarks	Multi-representation stacking	98.7% / 97.9%	Complex; paraphrase robustness
A6	Socolof & Kacholia	Stanford CS224N	DAIGT + HC3 + WELFake	Style embeddings + light heads	Fast, competitive AUC	Weak on hybrid text; not peer-reviewed
A7	M-DAIGT shared task	2025 / RANLP	M-DAIGT (≠ your data)	RoBERTa / ELECTRA / DeBERTa	Near-perfect F1	Saturated in-domain
A8	Zain et al.	2025 / RANLP	M-DAIGT	RoBERTa / TF-IDF-SVM / Candace	99.99% F1	Saturated; robustness open
A9	Yadagiri et al.	2025 / RANLP	M-DAIGT	DeBERTa + stylometry	Near-perfect	Cross-domain open
A10	Arun et al.	2026 / Springer	DAIGT V2 + variants	ML vs DL vs LLM	Comparative	Generalization
A11	zeyadusf — Catch the AI	— / GitHub	DAIGT	RoBERTa + DeBERTa fusion	—	Not peer-reviewed
A12	bitgrit blog	— / Medium	DAIGT V2	TF-IDF + LR/SGD/XGBoost	—	Not peer-reviewed
B1	**Guo et al. — HC3 paper**	**2023 / arXiv 2301.07597**	**HC3**	**GLTR, RoBERTa-single/QA**	**99.82 F1**	**No special prompts; unbalanced; OOD left as future work**
B2	Su et al. — HC3 Plus	2023 / arXiv 2309.02731	HC3 + semantic-invariant	Tk-Instruct detector	91.7% on hard tasks	ChatGPT-only (GPT-3.5-Turbo-0301)

B3	**Antoun et al.**	**2023 / TALN**	**HC3 (En + Fr)**	**RoBERTa/ELECTRA/XLM-R + attacks**	**99.88 F1 → 33.6% adversarial**	**No OOD generalization**
B4	**Tian et al. — MPU**	**2024 / ICLR**	**HC3 + TweepFake**	**Positive-Unlabeled short-text**	**58.6 → 85.3 F1**	**Documents whitespace artifact**
B5	Adversarial robustness	2024 / ACL 2406.01179	HC3 + TruthfulQA	Char perturbations	Large drop under attack	Limited attack range
B6	LLM-Detector	2024 / arXiv 2402.01158	HC3 (9 LLMs) + M4	LLM instruction tuning	Strong Chinese	Chinese-focused
B7	MMD-MP	2024 / arXiv 2402.16041	HC3 + XSum	Two-sample MMD test	Stable multi-generator	Complex
B8	Ardeshirifar	2025 / Springer AI&Ethics	HC3 + GPT-2	Handcrafted vs DL	DL wins; both drop cross-model	Standardizes punctuation artifact
B9	WPI thesis	— / WPI	HC3	RoBERTa fine-tune	High in-dom.	Not peer-reviewed
B10	Linguistic + transformer	2025 / Springer KAIS	HC3 + M4GT	DistilBERT + linguistics	99.45% HC3	Saturation
B11	**Baidya et al. — benchmark**	**2026 / arXiv 2603.17522**	**HC3 + ELI5**	**Broad benchmark + humanization**	**Near-perfect ID, drops OOD**	**Length confound**
B12	**Kim et al. — shortcuts**	**2024 / arXiv 2406.16275**	**HC3**	**FAILOpt shortcut probing**	**Detectors use shortcuts**	**Prompt-shortcut critique**
B13	Comm. in Statistics	2025 / T&F	HC3	BoW / TF-IDF / doc2vec + ML	High in-dom.	Classical baseline
B14	SICO — guided evasion	2023 / arXiv 2305.10847	HC3	Prompt-based evasion	Detectors evaded	Motivates robustness
C4	**Confounding Neurons**	**2025 / EMNLP Findings**	**DAIGT + HC3 + M4 + XSum**	**Neuron ablation**	**+6.9% OOD from removing 20 neurons**	**Detectors memorize spurious features**

Bold rows are the papers you should read first and cite most heavily.

---

7. References

Peer-reviewed — your citable core

1. Guo, B. et al. (2023). *How Close is ChatGPT to Human Experts? Comparison Corpus, Evaluation, and Detection.* arXiv:2301.07597. https://arxiv.org/abs/2301.07597 — Code: https://github.com/Hello-SimpleAI/chatgpt-comparison-detection
2. Su, Z. et al. (2023). *HC3 Plus: A Semantic-Invariant Human ChatGPT Comparison Corpus.* arXiv:2309.02731. https://arxiv.org/abs/2309.02731
3. Antoun, W., Mouilleron, V., Sagot, B., Seddah, D. (2023). *Towards a Robust Detection of Language Model Generated Text: Is ChatGPT that Easy to Detect?* TALN/CORIA 2023. arXiv:2306.05871. https://arxiv.org/abs/2306.05871 — Code: https://gitlab.inria.fr/wantoun/robust-chatgpt-detection
4. Tian, Y., Chen, H., Ho, et al. (2024). *Multiscale Positive-Unlabeled Detection of AI-Generated Texts.* ICLR 2024 (Spotlight). arXiv:2305.18149. https://arxiv.org/pdf/2305.18149 — Code: https://github.com/YuchuanTian/AIGC_text_detector
5. *Are AI-Generated Text Detectors Robust to Adversarial Perturbations?* (2024). ACL 2024. arXiv:2406.01179. https://arxiv.org/pdf/2406.01179
6. Kim, et al. (2024). *Investigating the Influence of Prompt-Specific Shortcuts in AI Generated Text Detection.* arXiv:2406.16275. https://arxiv.org/pdf/2406.16275
7. *How to Generalize the Detection of AI-Generated Text: Confounding Neurons.* (2025). Findings of EMNLP 2025. https://aclanthology.org/2025.findings-emnlp.1388.pdf
8. Ansary, Md. S. (2026). *A Multirepresentation Stacked Ensemble for AI-Generated Text Detection.* International Journal of Intelligent Systems (Wiley). https://onlinelibrary.wiley.com/doi/10.1155/int/3992539
9. Ardeshirifar, R. (2025). *Comparing hand-crafted and deep learning approaches for detecting AI-generated text: performance, generalization, and linguistic insights.* AI and Ethics (Springer). https://link.springer.com/article/10.1007/s43681-025-00699-4
10. *Synergizing linguistic features and transformer networks for detecting AI-generated text.* (2025). Knowledge and Information Systems (Springer). https://link.springer.com/article/10.1007/s10115-025-02637-6

11. *Detecting AI-generated texts using machine learning models.* (2025). Communications in Statistics: Case Studies. https://www.tandfonline.com/doi/full/10.1080/23737484.2025.2550442
12. Zhou, C. (2024). *Exploiting Machine Learning Model Ensemble for AI-Generated Texts Detection.* TCSISR vol. 5 (AIDML 2024). https://wepub.org/index.php/TCSISR/article/view/2382
13. Arun, S., Purohit, S., Pareek, J. (2026). *Analyzing AI-Generated Text Using Machine Learning, Deep Learning and Large Language Models.* Springer. https://link.springer.com/chapter/10.1007/978-3-032-00350-8_13
14. Lamsiyah, S. et al. (2025). *M-DAIGT: A Shared Task on Multi-Domain Detection of AI-Generated Text.* RANLP 2025. arXiv:2511.11340. https://arxiv.org/abs/2511.11340
15. Zain, Farooqui, Rafi (2025). *A Multi-Strategy Approach for AI-Generated Text Detection.* RANLP 2025. arXiv:2509.00623. https://arxiv.org/abs/2509.00623
16. Yadagiri, Sai Teja, Pakray, Chunka (2025). *AI-Generated Text Detection Using DeBERTa with Auxiliary Stylometric Features.* RANLP 2025. https://aclanthology.org/2025.ranlp-mdaigt.2/

Preprints — cite with appropriate caution

17. Alikhanov, Amangeldi, Demeubay, Akhmetzhan, Polat, Zharas, Moldakhmetov (2026). *AI Generated Text Detection.* arXiv:2601.03812. https://arxiv.org/abs/2601.03812
18. Baidya, et al. (2026). *Detecting the Machine: A Comprehensive Benchmark of AI-Generated Text Detectors.* arXiv:2603.17522. https://arxiv.org/abs/2603.17522
19. *Adaptive Ensembles of Fine-Tuned Transformers for LLM-Generated Text Detection.* (2024). arXiv:2403.13335. https://arxiv.org/abs/2403.13335
20. *LLM-Detector: Improving AI-Generated Chinese Text Detection with Open-Source LLM Instruction Tuning.* (2024). arXiv:2402.01158. https://arxiv.org/pdf/2402.01158
21. *Detecting Machine-Generated Texts by Multi-Population Aware Optimization for Maximum Mean Discrepancy.* (2024). arXiv:2402.16041. https://arxiv.org/pdf/2402.16041
22. *Large Language Models can be Guided to Evade AI-Generated Text Detection.* (2023). arXiv:2305.10847. https://arxiv.org/pdf/2305.10847

Not peer-reviewed — engineering references only

23. Biswas, R. et al. (2024). *1st Place Solution, Kaggle LLM — Detect AI Generated Text.* https://github.com/rbiswasfc/llm-detect-ai
24. Socolof, G. Z., Kacholia, R. *Fast, Interpretable AI-Generated Text Detection Using Style Embeddings.* Stanford CS224N. https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/final-projects/GiuliaZoeSocolofRitikaKacholia.pdf
25. *Supervised Machine Generated Text Detection Using LLM.* WPI MS thesis. https://digital.wpi.edu/downloads/hh63t0231
26. zeyadusf. *DAIGT — Catch the AI.* https://github.com/zeyadusf/DAIGT-Catch-the-AI
27. Neo, B. *Did AI Write This?* bitgrit. https://medium.com/bitgrit-data-science-publication/did-ai-write-this-7963a8d7ed2d

Datasets

28. *DAIGT V2 Train Dataset* (thedrcat). https://www.kaggle.com/datasets/thedrcat/daigt-v2-train-dataset
29. *LLM — Detect AI Generated Text* (Kaggle competition). https://www.kaggle.com/competitions/llm-detect-ai-generated-text
30. *PERSUADE 2.0 Corpus.* https://github.com/scrosseye/persuade_corpus_2.0
31. *HC3 — Human ChatGPT Comparison Corpus.* https://huggingface.co/datasets/Hello-SimpleAI/HC3

---

8. Caveats on this report

Non-peer-reviewed sources are labelled. Items 23–27 are Kaggle write-ups, GitHub projects, a course report, and a thesis. Use them for implementation guidance, never as citations of record.

"DAIGT" is ambiguous. Your dataset is the Kaggle competition plus thedrcat's V2 train set. M-DAIGT (papers A7–A9) is a separate RANLP 2025 dataset of news and academic abstracts. Cite the latter only as related work in the broader family.

HC3 counts vary between papers. You will see 24,322 questions, "40K questions", 23,363 pairs, and ~74k answers — because authors count questions, answers, or paired samples differently and use different filtered versions. Always state which version and count you used.

Very recent preprints. Some arXiv identifiers (2601.xxxxx, 2603.xxxxx) are recent preprints. Check for updated versions before citing, and treat non-peer-reviewed preprints accordingly.

Numbers are as reported. No model was re-run for this report. All figures come from the cited authors. Verify the DAIGT V2 counts against the current CSV before publishing — the Kaggle dataset card has been revised across versions.

Verify links before submission. Every link here was checked at the time of writing, but URLs decay. Re-check before your bibliography goes out.